

UNITÉ 8 · SEMAINES 13–14

Réseau: les sockets

📅 Semaines 13–14 · dates à confirmer </> Laboratoire TD7 (corrigé + défi IA) ~ 2 animations

LABORATOIRE TD7 · CORRIGÉ

Le réseau avec les sockets: d'un serveur minimal à un serveur web à cinq modèles

Un socket s'utilise comme un fichier (`read`, `write`, `close`), mais il relie des processus sur des machines différentes. Ce laboratoire rassemble tout: `fork` par client, un thread par connexion et le producteur–consommateur du TD6 pour servir de nombreux clients à la fois.



Devoir : rapport du TP

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|--|---|--|
| 1 | Le strict minimum d'un serveur | 3 | Serveur web: itératif, un thread par connexion et pool |
| 2 | Serveur d'écho, itératif et avec <code>fork</code> | 4 | Exploration: états TCP, file de <code>listen</code> , modèles sous charge et UDP |

TD7 · 1

● Corrigé · testé

Le strict minimum d'un serveur

Énoncé. Un programme qui attend une connexion (testez-le avec `telnet` ou `nc`) et se termine: `socket` (IPv4, orienté connexion) → `sockaddr_in` avec `INADDR_ANY` et le port dans l'ordre réseau (`htons`) → `bind` → `listen` → `accept` → `close` .

Solution



C tds/td7/sock.c

```

/*
 * TD7 - exercise 1: the server side of a TCP connection, step by step.
 * ./sock [port]      (default 7000)  then, from another terminal:
telnet localhost 7000
 * socket → bind → listen → accept → close.
 */
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000

int main(int argc, char *argv[])
{
    int port = argc > 1 ? atoi(argv[1]) : PORT;

    /* 1. An IPv4 (AF_INET), connected, reliable byte stream (SOCK_STREAM =
TCP). */
    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");

    /* Allow restarting the server right away (otherwise bind fails for ~60
s: TIME_WAIT). */
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");

    /* 2. Local address: any interface of this machine, the chosen port
(network byte order). */
    struct sockaddr_in addr = { 0 };
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);

    /* 3. and 4. Bind the address, start listening (queue of 1 pending
connection). */
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 1) == -1, "listen");
    printf("listening on port %d\n", port);

```

```
fflush(stdout);

/* 5. accept blocks until a client connects and returns a NEW descriptor
for it. */
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");

/* 6. Close the connection and the listening socket. */
exit_if(close(client) == -1 || close(s) == -1, "close");
return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./sock 37634 & sleep 0.3; nc -z 127.0.0.1 37634 && echo 'client connected';
wait
listening on port 37634
connection accepted, closing
Connection to 127.0.0.1 37634 port [tcp/*] succeeded!
client connected
```

- `accept` renvoie un **nouveau descripteur** pour ce client; le socket d'écoute reste disponible pour en accepter d'autres.
- `htons` / `htonl` : le réseau utilise l'ordre *big-endian* ; x86 est *little-endian* (vous l'avez vu dans le TD1 avec `od`).
- `SO_REUSEADDR` permet de relancer le serveur tout de suite: sans lui, `bind` échoue avec `EADDRINUSE` tant que les connexions précédentes sont en `TIME_WAIT`.

Animation A8.1 · Cycle de vie d'une connexion →

Serveur d'écho, itératif et avec `fork`

Énoncé. Un serveur d'écho renvoie au client tout ce qu'il reçoit, sauf `quit`, qui ferme la connexion et en attend une autre. Lancez plusieurs clients à la fois: que se passe-t-il? Modifiez le serveur avec `fork` pour en servir plusieurs.

Solution



C tds/td7/echo_server.c

```

/*
 * TD7 - exercise 2: echo server. Sends back every line it receives; "quit"
 closes the
 * connection and the server waits for the next client.
 * ./echo_server [port] [-f]      -f: one child process per client (fork)
 * Without -f the server is ITERATIVE: a second telnet waits until the first
 one leaves
 * (its connection stays in the listen queue). With -f each client gets its
 own process.
 */
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000
#define LINE 1024

/* Echo line by line until end of file or "quit". Returns when the client is
done. */
static void serve(int c)
{
    char line[LINE];
    size_t len = 0;

    for (;;) {
        ssize_t n = read(c, line + len, 1);          /* byte by byte: simple,
fine for a lab */
        if (n == -1 && errno == EINTR)
            continue;
        if (n <= 0)                                  /* 0: the client
closed; -1: error */
            return;
        if (line[len] == '\n' || len == LINE - 2) {

```

```

        len++;
        if (strncmp(line, "quit", 4) == 0)
            return;
        if (write_all(c, line, len) == -1)
            return;
        len = 0;
    } else {
        len++;
    }
}

}

int main(int argc, char *argv[])
{
    int port = PORT, use_fork = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) use_fork = 1;
        else port = atoi(argv[i]);
    }

    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htonl((uint16_t)port),
                                .sin_addr.s_addr = htonl(INADDR_ANY) };
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 8) == -1, "listen");
    printf("echo server on port %d (%s)\n", port, use_fork ? "one process per
client" : "iterative");
    fflush(stdout);

    /* Children that finish must not stay zombies: ignoring SIGCHLD makes the
kernel reap them. */
    if (use_fork)
        exit_if(signal(SIGCHLD, SIG_IGN) == SIG_ERR, "signal");

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c == -1 && errno == EINTR)
            continue;
        exit_if(c == -1, "accept");
    }
}

```

```

    if (!use_fork) {
        serve(c);
        exit_if(close(c) == -1, "close");
        continue;
    }
    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        exit_if(close(s) == -1, "close");          /* the child does not
accept */
        serve(c);
        _exit(EXIT_SUCCESS);
    }
    exit_if(close(c) == -1, "close");              /* the parent does not
talk to the client */
    }
}

```

Sortie réelle

```

$ python3 tcp_client.py echo 22751 'hola\nmundo\nquit'
sent: hola      received: hola
sent: mundo     received: mundo

```

Sortie réelle

```

$ python3 tcp_client.py blocked 22751      # iterative server
client B got nothing after 1 s: the server is busy with client A

```

Sortie réelle

```

$ python3 tcp_client.py blocked 34885      # server with fork
client B got 'hi\n' after 0.00 s

```

Le serveur itératif sert un client à la fois: pendant que le client A est connecté, B attend dans la file de `listen` et ne reçoit aucune réponse (le test le mesure: rien en 1 s). Avec `-f`, chaque connexion a son propre processus et B reçoit son écho tout de suite. Le parent ferme le socket du client et l'enfant celui d'écoute; ignorer `SIGCHLD` évite les zombies. La version héritée détectait `quit` avec `strstr` sur un tampon sans terminateur; elle lit désormais ligne par ligne.

Serveur web: itératif, un thread par connexion et un pool de 8 threads

Énoncé. Le client (un navigateur) envoie `GET fichier` ; le serveur répond `HTTP/1.0 200 OK`, le type MIME et le contenu. Utilisez `stat` pour savoir s'il existe et sa taille, `mmap` pour le projeter et `write` pour l'envoyer. Ensuite: (1) un nouveau thread par connexion; (2) 8 threads créés au démarrage qui attendent des connexions — le serveur *produit* des connexions et les threads les *consomment*.

Solution

C tds/td7/http_server.c

```
/* Handles one request on the connected socket c. Never exits the process. */
static void handle(int c)
{
    char req[REQUEST], path[1024], full[2048];
    ssize_t n = read(c, req, sizeof req - 1);
    if (n ≤ 0)
        return;
    req[n] = '\0';

    if (sscanf(req, "GET %1023s", path) ≠ 1) {
        reply_error(c, "400 Bad Request");
        return;
    }
    if (strstr(path, "..") ≠ NULL) {                                /* no escape from root */
        reply_error(c, "403 Forbidden");
        return;
    }
    if (strcmp(path, "/") = 0)
        strcpy(path, "/index.html");
    snprintf(full, sizeof full, "%s%s", root, path);

    struct stat st;
    if (stat(full, &st) = -1 || !S_ISREG(st.st_mode)) {
        reply_error(c, "404 Not Found");                            /* PG204 answered
"201" by mistake */
        return;
    }
    int fd = open(full, O_RDONLY);
    if (fd = -1) {
```

```

        reply_error(c, "403 Forbidden");
        return;
    }
    char head[256];
    int hn = snprintf(head, sizeof head,
                      "HTTP/1.0 200 OK\r\nContent-Type: %s\r\nContent-Length:
%lld\r\n\r\n",
                      mime_type(full), (long long)st.st_size);
    if (write_all(c, head, (size_t)hn) == -1) {
        close(fd);
        return;
    }
    if (st.st_size > 0) {
        /* mmap of 0 bytes is
an error */
        void *data = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE,
fd, 0);
        if (data != MAP_FAILED) {
            (void)write_all(c, data, (size_t)st.st_size); /* loops over
partial writes */
            munmap(data, (size_t)st.st_size);
        }
    }
    close(fd);
}

/* ----- thread per connection ----- */
static void *connection_thread(void *arg)
{
    int c = (int)(intptr_t)arg;
    handle(c);
    close(c);
    return NULL;
}

/* ----- pool: bounded queue of connected sockets ----- */
static int queue[QUEUE];
static int q_in = 0, q_out = 0;
static sem_t q_free, q_used;
static pthread_mutex_t q_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg)
{
    (void)arg;
    for (;;) {
        sem_wait(&q_used);
        /* wait for a connection */

```

```
pthread_mutex_lock(&q_lock);
int c = queue[q_out];
q_out = (q_out + 1) % QUEUE;
pthread_mutex_unlock(&q_lock);
sem_post(&q_free);
handle(c);
close(c);
}
return NULL;
}
```

Sortie réelle

```
$ curl -s -i http://localhost:PORT/ | tr -d '\r' # mode pool
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 20

<h1>FS0 · UTI</h1>
```

Sortie réelle

```
$ curl -s -o /dev/null -w '%{http_code}' ../missing.html ; curl --path-as-is
.../../etc/passwd
404
403
```

MODE	COMMENT IL SERT	LIMITE
<code>-m iterative</code>	Un client à la fois	Un client lent bloque tout le monde
<code>-m fork</code>	Un processus enfant par connexion	Créer un processus coûte plusieurs fois plus qu'un thread (mesuré dans le TD6 · 5)
<code>-m thread</code>	Un nouveau thread par connexion (<code>pthread_detach</code>)	Des milliers de connexions = des milliers de threads (mémoire, changements de contexte)
<code>-m pool</code>	8 threads fixes + file bornée (sémaphores + mutex)	Ressources bornées; 8 clients lents le saturent (TD7 · 4)
<code>-m epoll</code>	Un thread et une boucle d'événements: il ne sert que les sockets qui ont déjà des données	Cette version simple écrit encore de façon bloquante; nginx ou Node.js écrivent aussi sans bloquer

Les tests envoient 40 requêtes concurrentes (8 à la fois) dans chaque mode et vérifient 40 réponses 200 et un binaire identique à l'original. Sécurité minimale: un chemin avec `..` reçoit **403** (le client ne peut pas sortir du dossier servi) et ce qui n'existe pas, **404**. Le serveur ignore SIGPIPE pour ne pas mourir si un client se déconnecte au milieu de la réponse.

⚠ Corrigé et complété

La version héritée répondait `201 Not found` (le bon code est 404), ne bloquait pas `..`, n'envoyait pas de `Content-Length`, ignorait les écritures partielles, ne vérifiait pas l'ouverture du fichier et n'avait pas les versions `thread` et `pool` que demande l'énoncé.

Animation A8.2 · Modèles de serveur →

Exploration: ce que le noyau fait de vos sockets

Énoncé. (1) Lisez avec `getsockopt(TCP_INFO)` l'état de chaque socket à chaque étape d'une connexion. (2) Découvrez ce que limite le backlog de `listen`. (3) Mesurez les cinq modes du serveur web avec des clients lents. (4) Comparez TCP et UDP. (5) Vérifiez le bogue de `htons` des diapositives d'origine et résolvez des noms avec `getaddrinfo`. (6) Créez un socket local.

1 · États TCP (`tcp_states.c`)

C tds/td7/tcp_states.c

```
struct tcp_info ti;
socklen_t len = sizeof ti;
exit_if(getsockopt(s, IPPROTO_TCP, TCP_INFO, &ti, &len) == -1,
"getsockopt");
return ti;
}

static const char *state(int s)
{
    if (s == -1)
        return "-";
    unsigned st = info(s).tcpi_state;
    return st < sizeof NAMES / sizeof NAMES[0] ? NAMES[st] : "?";
}

/* One line per step: the state of each socket. For a listening socket the
kernel reports the
* accept queue in tcpi_unacked (connections waiting for accept) and
tcpi_sacked (backlog). */
static void show(const char *step, int l, int c, int a)
{
    struct tcp_info li = info(l);
    printf("%-26s | listener %s (queue %u/%u) | client %-11s | server %s\n",
        step, state(l), li.tcpi_unacked, li.tcpi_sacked, state(c),
        state(a));
}
```

Sortie réelle

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -
| server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED
| server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED
| server ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2
| server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED
| server CLOSED
                           | the client's end moved to a TIME_WAIT entry of
the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Trois faits que montre la sortie: la connexion est **établie avant** `accept` (le noyau fait la poignée de main; la file passe à 1/4); celui qui ferme en premier passe par `FIN_WAIT2` pendant que l'autre reste en `CLOSE_WAIT` jusqu'à ce que son `read` renvoie 0; et après la fermeture, le noyau garde une entrée **TIME_WAIT** (60 s) que `ss` montre alors que le programme est terminé. C'est pourquoi les serveurs utilisent `SO_REUSEADDR`.

2 · Ce que limite le backlog (`backlog.c`)

Sortie réelle

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5,
SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8,
SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

Avec `listen(fd, 4)`, Linux termine **backlog + 1 = 5** poignées de main et jette le SYN des autres, qui restent en `SYN_SENT` et le **renvoient** (1 s, puis 2 s, 4 s...). Quand le serveur accepte, la file se vide et les renvois entrent. Le backlog ne limite pas le

nombre de clients que sert le serveur: il limite le nombre de connexions terminées qui peuvent attendre `accept`.



Pour les curieux: les SYN cookies

Sortie réelle

```
$ nstat -n; ./backlog 2 8; nstat | grep -E
'Syncookies|ListenOverflows'
listen(2), 8 clients connect
0.3 s, no accept yet:          accept queue 3 | clients
ESTABLISHED 3, SYN_SENT 5
accept 3 (total 3), wait:      accept queue 3 | clients
ESTABLISHED 8, SYN_SENT 0
accept 3 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
accept 0 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
TcpExtSyncookiesSent 2
TcpExtListenOverflows 7
```

Avec `listen(fd, 2)`, il se passe quelque chose de plus subtil: quand la file des poignées de main à moitié faites se remplit aussi, le noyau répond avec des **SYN cookies** (sans garder d'état, une défense contre les inondations de SYN). Si la file d'acceptation est pleine quand l'ACK final arrive, cet ACK est perdu: le client se croit ESTABLISHED, mais le serveur n'en sait rien. Les compteurs du noyau (`nstat`) le confirment. C'est pourquoi le backlog d'un vrai serveur est grand (sur cette machine, `net.core.somaxconn = 4096`).

3 · Les cinq modèles sous charge (`load.c`)



32 clients: d'abord 1 (puis 8) qui mettent 1 s à envoyer leur requête, comme un client sur un mauvais réseau ou une attaque *slowloris*; 20 ms plus tard, les rapides. On mesure la latence de chaque groupe, de `connect` jusqu'à la réponse complète.

Sortie réelle

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 982 ms, max 983 ms
```

Sortie réelle

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 981 ms, max 981 ms
```

Sortie réelle

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 1 ms, max 2 ms
```

Itératif: un seul client lent fait attendre tout le monde ~1 s. Pool de 8: il tient avec 1 client lent, mais avec 8, les huit threads sont bloqués dans `read` et la même chose se reproduit. `fork`, un thread par client et `epoll` répondent aux rapides en quelques millisecondes (les sorties `fork` et `thread` sont dans `tests/out/td7/`). La boucle d'événements le fait avec **un seul thread** :

C tds/td7/http_server.c

```
/* ----- event loop: one thread watches every socket ----- */
static void event_loop(int s)
{
    int ep = epoll_create1(0);
    exit_if(ep == -1, "epoll_create1");
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = s }, ready[64];
    exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, s, &ev) == -1, "epoll_ctl");
    for (;;) {
        int n = epoll_wait(ep, ready, 64, -1); /* sleeps until some socket
is readable */
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "epoll_wait");
        for (int i = 0; i < n; i++) {
            int fd = ready[i].data.fd;
            if (fd == s) { /* a new connection: watch
it too */
                int c = accept(s, NULL, NULL);
                if (c == -1)
                    continue;
                ev.data.fd = c;
                exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, c, &ev) == -1,
"epoll_ctl");
            } else { /* the request has arrived:
read won't wait */
                exit_if(epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL) == -1,
"epoll_ctl");
                handle(fd);
                close(fd);
            }
        }
    }
}
```

Animation A8.2 · Modèles de serveur →

4 · TCP contre UDP (`udp.c`)



Sortie réelle

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3
"two", recvfrom #3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8
received, 9992 lost
      kernel counter Udp RcvbufErrors grew by 9992
```

TCP est un **flux d'octets** : trois `write` arrivent en un seul `read` (comme avec un tube), donc le protocole applicatif doit marquer où finit chaque message. UDP conserve les **frontières** de chaque datagramme, mais n'attend pas le récepteur : avec le tampon plein, le noyau jette des datagrammes sans prévenir personne, même au sein d'une même machine, et ne le note que dans `/proc/net/snmp`.

5 · Adresses et ports (`addr.c`) · 6 · Socket local (`local.c`)

Sortie réelle

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110,
sockaddr_storage 128
```

Sortie réelle

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path
holds 108 bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

L'exemple de la diapositive 187 mettait `sin_port = 7000` sans `htons` : sur un processeur *little-endian*, les deux octets sont lus à l'envers et le noyau ouvre le port **22555**. `gethostbyname` (aussi sur cette diapositive) ne connaît qu'IPv4, n'est pas sûr avec les threads et a quitté POSIX en 2008 : `getaddrinfo` résout IPv4 et IPv6 avec le même appel. Un socket `AF_UNIX` n'utilise pas 127.0.0.1 (c'est `AF_INET` sur l'interface de bouclage) : son adresse est un **fichier de type socket**, et le noyau dit au serveur qui s'est connecté (`SO_PEERCRED`), ce dont se servent systemd, D-Bus ou Docker pour décider des permissions.

Animation A8.1 · Cycle de vie d'une connexion →

Liste de vérification

Défi avec l'IA générative · Un serveur `epoll` qui ne se bloque jamais

Objectif. Transformez le mode `-m epoll` de `http_server.c` (qui lit et écrit encore de façon bloquante) en une vraie boucle d'événements, avec des sockets non bloquants et des écritures partielles, et mesurez-le face au mode un thread par client avec des milliers de connexions.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

J'ai un serveur HTTP en C avec cinq modes ; le mode `epoll` utilise un seul thread, mais `handle()` fait un `read` et un `write` qui peuvent bloquer [coller `http_server.c` et `load.c`]. Aide-moi à : (1) utiliser `accept4(..., SOCK_NONBLOCK)` et garder un état par connexion (lire la requête jusqu'à `\r\n\r\n`, écrire la réponse avec un décalage), en n'enregistrant `EPOLLOUT` que lorsque `write` renvoie `EAGAIN` ; (2) étendre `load.c` pour ouvrir 2000 connexions qui envoient très lentement ; (3) comparer le mode `epoll` avec `-m thread` en mesurant la latence des clients rapides, le nombre de threads (`ps -o nlwp`) et la mémoire (`ps -o rss`). Explique-moi le problème C10K et quelle ressource du système limite chaque mode.

Comment vérifier le résultat

- Un client qui lit un gros fichier très lentement (`curl --limit-rate 10k`) ne retarde pas les autres : `./load` le montre.
- `tests/run.sh td7` passe toujours et le fichier binaire arrive identique (`cmp`).
- Le tableau vient de vos mesures et indique sur quelle machine elles ont été faites.
- `ulimit -n` explique au moins un échec que vous observez en augmentant la concurrence.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.